
Security Review Report
NM-0998 - VOICE - VOTE SYSTEM



NETHERMIND
SECURITY

(August 20, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	System Actors	4
4.2	Poll Creation and Voting Lifecycle	4
4.3	Reward Commitment and Claims	4
4.4	Withdrawals and Account Cleanup	4
4.5	Token Validations and Promotional Rewards	5
5	Risk Rating Methodology	6
6	Issues	7
6.1	[Info] Flawed recommit logic can lead to double funding	7
6.2	[Info] Price cliff in poll creation fee allows users to pay less for 1 day Polls	7
7	Documentation Evaluation	8
8	Test Suite Evaluation	9
8.1	Tests Output	9
9	About Nethermind	13

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for Voice protocol's token-based voting system on the Solana blockchain. The primary focus of the engagement has been the `vote_system` Anchor program and its mechanisms for managing user deposits, off-chain reward commitments, and token validation.

Voice is a polling protocol that isolates user principal from system payout mechanisms by utilizing a two-vault architecture for every poll. Voting requires users to deposit base tokens into a dedicated `poll_vault`, while a trusted off-chain **Rewarder** funds a separate `poll_reward_vault` upon publishing a Merkle root containing the final reward distribution. The program implements a permissionless pull-based model, allowing anyone to execute `claim_reward` by presenting a valid cryptographic proof against the committed root, while ensuring users can withdraw their original deposits via `withdraw_vote` based purely on a poll's expiration time.

The audit comprises approximately 2386 lines of Rust code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 2 point of attention, where the issues are classified as Informational severity. The issue is summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

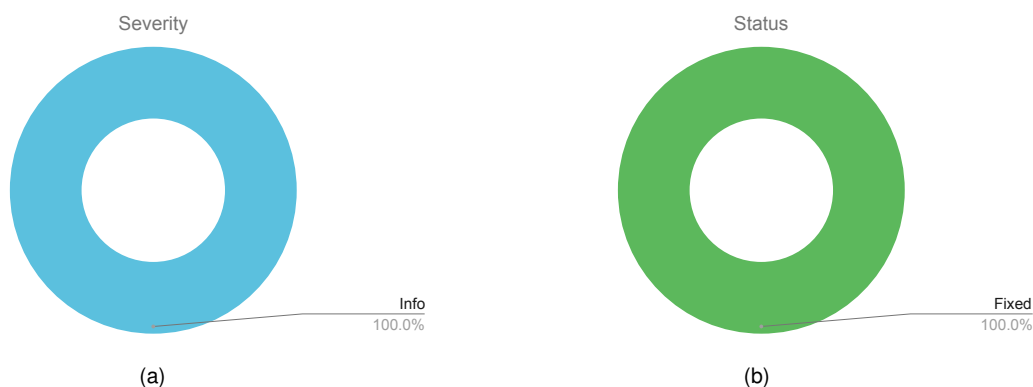


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (0), Undetermined (0), Informational (2), Best Practices (0). Distribution of status: Fixed (2), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	Aug 10, 2026
Final Report	Aug 20, 2026
Initial Commit	cde83d8
Final Commit	bfc65bf
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	File	LoC	Comments	Ratio	Blank	Total
1	programs/vote-system/src/validate_mint_tests.rs	257	39	15.2%	29	325
2	programs/vote-system/src/merkle.rs	275	78	28.4%	61	414
3	programs/vote-system/src/lib.rs	1854	576	31.1%	331	2761
	Total	2386	693	29.0%	421	3500

3 Summary of Issues

	Finding	Severity	Update
1	Flawed recommit logic can lead to double funding	Info	Fixed
2	Price cliff in poll creation fee allows users to pay less for 1 day Polls	Info	Fixed

4 System Overview

Voice is a token-based voting system deployed on the Solana blockchain and built utilizing the Anchor framework. The protocol enables users to participate in decentralized polls by depositing SPL or Token-2022 tokens into isolated, per-poll vaults. An architectural characteristic of Voice is its hybrid execution model: while voting and fund custody are handled on-chain, reward allocations are computed off-chain by a designated backend service and subsequently pushed on-chain via a Merkle root.

4.1 System Actors

The protocol defines several roles with specific permissions and responsibilities:

- **Admins:** maintain global oversight of the protocol via the SystemCore account. They configure the system's parameters such as claim windows, maximum poll durations, and daily reward budgets. Furthermore, admins hold the authority to disable polls, shorten active poll durations via `edit_poll(...)`, and perform reward root corrections before claims begin.
- **Rewarder:** The Rewarder is a trusted off-chain entity responsible for computing the reward distributions based on poll outcomes. This actor interacts with the chain to publish Merkle roots and fund the reward vaults through the `commit_rewards_root(...)` function.
- **Creators:** Creators initiate new polls, define their operational timeframe (start and end times), and pay a required creation fee.
- **Users (Voters):** Users interact with active polls by locking tokens in exchange for voting power. They are responsible for retrieving their own deposits once a poll ends and can claim their allocated rewards using Merkle proofs.
- **Promo Funder:** An optional role permitted to distribute promotional rewards directly to users alongside the Admin.

4.2 Poll Creation and Voting Lifecycle

The lifecycle of a poll begins with its initialization and continues through an active voting phase.

- **Poll Initialization:** A creator invokes `create_poll(...)` to instantiate a new `Poll` account. This process assesses a dynamic creation fee that is split 15% to the Admin and 85% to the Rewarder, with a 10% discount automatically applied for polls lasting longer than one day.
- **Vault Segregation:** Upon creation, the protocol derives two distinct Program Derived Address (PDA) vaults for the poll. The `poll_vault` is dedicated exclusively to holding user deposits, while the `poll_reward_vault` is reserved strictly for payout funds.
- **Voting Mechanics:** Users cast votes by calling `make_vote(...)`, which transfers base tokens from the user's associated token account into the `poll_vault`. A user's voting choice (variant 1 or 2) is permanently locked upon their first deposit for that poll. However, a user may repeatedly call the function to top-up their voting power before the poll concludes.

4.3 Reward Commitment and Claims

The protocol utilizes a pull-based Merkle tree architecture to distribute rewards.

- **Reward Commitment:** After a poll reaches its `end_time`, the Rewarder calls `commit_rewards_root(...)` to publish a Merkle root representing the complete payout set for both voters and the creator. In this same transaction, the Rewarder transfers the total reward amount from their own account into the `poll_reward_vault`.
- **Daily Budget Limits:** To bound the impact of a compromised Rewarder key, commitments are constrained by a `daily_reward_budget` tracked inside the SystemCore account.
- **Permissionless Claims:** Users execute `claim_reward(...)` or `claim_creator_reward(...)` by providing a valid Merkle proof. The claim validates the proof against the committed root and transfers the tokens from the `poll_reward_vault` to the recipient's ATA. Claims require no signature from the recipient because the proof binds both the amount and the destination account. Furthermore, the system lacks a global pause mechanism for claims; once claims have started, neither the Admin nor the Rewarder can halt the process or recommit a new root.

4.4 Withdrawals and Account Cleanup

The recovery of deposits and the management of stale state are treated as entirely independent operational paths.

- **Deposit Withdrawals:** The extraction of principal deposits via `withdraw_vote(...)` is time-gated. As soon as the poll's `end_time` is reached, a voter can retrieve their tokens from the `poll_vault`. This mechanism is structurally isolated from the reward state, meaning voters can recover their principal regardless of whether rewards were ever calculated, committed, or claimed.
- **Reward Sweeping:** Rewards must be claimed within a snapshotted claim window. Once the `effective_close_deadline` passes, any unclaimed funds remaining in the `poll_reward_vault` can be permissionlessly swept back to the Rewarder via `close_reward_vault(...)`.
- **State Closure:** To reclaim rent, fully settled `Vote` accounts can be closed after deposits are withdrawn and rewards are either claimed or forfeited. Similarly, an empty `poll_vault` can be closed permissionlessly after the poll ends, returning the stored rent to the original poll creator.

4.5 Token Validations and Promotional Rewards

Voice employs validation for the tokens it supports and offers a parallel distribution channel for promotional campaigns.

- **Mint Vetting:** During system initialization, the `validate_mint(...)` function checks the chosen token mint. The protocol rejects mints possessing a `freeze_authority`, which could otherwise be abused to lock the deposit vaults. For Token2022 mints, the system utilizes an allowlist of extensions, rejecting any features that alter transfer mechanics or allow account pausing.
- **Promo Mechanism:** Independent of the poll reward logic, Admins and Promo Funders can invoke `distribute_promo_reward(...)` to allocate funds directly to a user's `UserProfile` account. These tokens are stored in the global `system_core_ata` and can be retrieved by users at any time using the `withdraw_promo(...)` function.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Info] Flawed recommit logic can lead to double funding

File(s): `src/lib.rs`

Description: The `commit_rewards_root(...)` function allows the admin to correct a committed reward root by replacing it and updating the total reward amount. However, if an admin performs a downward correction, the surplus remains in the vault but the `rewards_total` is lowered. If the admin later performs an upward correction, the required funding is calculated against the lowered `rewards_total` rather than the actual vault balance. This causes the admin to unnecessarily double-fund the vault for the difference, which will also consume more from the `daily_reward_budget` and causes a temporary freezing of funds for the rewarder.

```

1  pub fn commit_rewards_root(...) -> Result<> {
2      // ...
3      let funding = if ctx.accounts.poll.rewards_committed_at == 0 {
4          total
5      } else {
6          // ...
7          // @audit-issue The funding is calculated using the tracked variable rather than the actual vault balance.
8          total.saturating_sub(ctx.accounts.poll.rewards_total)
9      };
10
11     if funding > 0 {
12         // @audit This consumes the daily budget based on the flawed funding calculation.
13         check_and_consume_daily_reward_budget(&mut ctx.accounts.system_core, now_ts, funding)?;
14         // ...
15     }
16     // ...
17 }

```

Recommendation(s): Consider calculating the required funding delta by checking the actual balance of the reward vault rather than the tracked `rewards_total` state variable.

Status: Fixed

Update from the client: Fixed at commit [35e679c](#)

6.2 [Info] Price cliff in poll creation fee allows users to pay less for 1 day Polls

File(s): `src/lib.rs`

Description: The `calculate_poll_creation_fee(...)` function calculates the fee required to create a new poll based on its duration. A 10% discount is applied to the total `base_cost` as soon as the duration strictly exceeds 1 day (86,400 seconds). This step-function logic breaks the pricing curve. For example, a poll lasting exactly 1 day costs full price, but a poll lasting 1 day and 1 second receives a 10% discount on its entire duration. Users are economically incentivized to slightly extend their 1-day polls to receive the discount, which means that the protocol will get fewer fees.

```

1  pub fn calculate_poll_creation_fee(...) -> Result<u64> {
2      // ...
3      let base_cost = duration_seconds
4          .checked_mul(price_per_second)
5          .ok_or(VoteError::ArithmeticOverflow)?;
6      if duration_seconds <= 86400 {
7          return Ok(base_cost);
8      }
9      // ...
10     // @audit-issue The discount is applied to the entire base cost if the duration exceeds 1 day.
11     let discount = base_cost
12         .checked_div(10) // 0.1
13         .ok_or(VoteError::ArithmeticOverflow)?;
14
15     let final_fee = base_cost
16         .checked_sub(discount)
17         .ok_or(VoteError::ArithmeticOverflow)?;
18     Ok(final_fee)
19 }

```

Recommendation(s): Consider applying the discount only to the portion of the duration that exceeds 1 day.

Status: Fixed

Update from the client: Fixed at commit [35e679c](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Voice's documentation

The Voice team provided a comprehensive overview of the main system components during the kick-off call with a detailed explanation of the intended functionalities along with a written specification for the different modules and roles. Additionally, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and in depth understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

anchor **test**

```
Unit: program_version()
  emits Cargo package version (13546ms)

Unit: initialize()
  rejects a caller who is not the program upgrade authority (1260ms)
  initializes system core + system core ATA + reward limits (408ms)
  rejects repeated initialize

Unit: transferOwnership() + acceptOwnership()
  transfers ownership and accepts it: admin -> new admin -> back (2080ms)
  admin can cancel a pending proposal (835ms)
  admin can overwrite a pending proposal with a new one (1255ms)
  non-admin cannot transfer ownership (451ms)
  non-admin cannot cancel a transfer (1226ms)
  wrong signer cannot accept transfer (1258ms)
  cannot accept when no transfer is pending (453ms)
  after transfer, old admin cannot call admin-only instructions (2069ms)
  new admin can call admin-only instructions (2937ms)
  fails with wrong systemCore address
  publishes an event for every step of the handover (2927ms)

Unit: setRewarder()
  admin can set rewarder (1256ms)
  non-admin cannot set rewarder (UnauthorizedAdmin) (449ms)
  fails if admin account is correct but NOT signed by admin (400ms)
  fails with wrong systemCore address

Unit: setPollCreationPrice()
  admin can set poll creation price (820ms)
  non-admin cannot set poll creation price (UnauthorizedAdmin) (449ms)
  fails if admin account is correct, but signer is not admin (413ms)
  fails with wrong systemCore address
  allows setting price to 0 (809ms)
  supports very large u64 values (837ms)

Unit: createPoll()
  creates poll + creator profile, charges fee (creator=admin=rewarder => net 0) (421ms)
  charges a real fee when creator != admin (creator=user, rewarder=admin => admin ATA increases) (1677ms)
  stores poll_is_disabled=true in poll account (417ms)
  creates creator profile for user (1679ms)
  creates poll for free when poll_creation_price_per_second = 0 (2512ms)
  rejects createPoll when fee calculation overflows (837ms)
  rejects end_time <= start_time
  rejects a poll whose end time has already passed
  fails if rewarderAta.owner != system_core.rewarder (616ms)
  fails if creatorProfile PDA does not belong to creator (2306ms)
  fails if adminAta.mint != mint (1062ms)
  fails if rewarderAta.mint != mint (832ms)
  fails if creatorAta.owner != creator (1041ms)
  fails if creatorAta.mint != mint (2320ms)

Units: editPoll() (shorten only)
  admin can shorten a poll, and nothing else changes (609ms)
  rejects extending a poll (422ms)
  rejects an unchanged end time (419ms)
  rejects shortening to at or before the start time (422ms)
  non-admin cannot edit a poll (1676ms)
  rejects editing once rewards are committed (3352ms)
  shortening into the past ends the poll: voting stops, the deposit unlocks (4189ms)
  edits only the targeted poll (1258ms)

unit: makeVote()
  creates vote + init user_profile, moves tokens to poll vault, updates counters (2104ms)
  same user can vote on two different polls (one vote account per poll) (2929ms)
  rejects makeVote with a poll vault derived from another poll (2111ms)
  allows a repeat vote (top-up): accumulates power, keeps choice and single vote account (2503ms)
```

VoteCreated reports base units, and its total tracks Vote.amount across a top-up (2555ms)
 rejects a top-up that changes the vote choice (2063ms)
 rejects `vote_power = 0` (1674ms)
 rejects `vote_power` that overflows token base-unit scaling (1677ms)
 rejects invalid `vote_choice` (0 or 4) (1683ms)
 rejects `vote_choice=3` (only two variants exist) (1669ms)
 fails `if voterAta.owner != voter` (1675ms)
 fails `if voterAta.mint != mint` (2738ms)
 fails `if userProfile PDA does not belong to voter` (2712ms)
 fails with wrong pollVault PDA (1680ms)
 rejects voting when poll is disabled (1672ms)
 rejects voting when poll is not active (start `time in` the future) (1685ms)
 rejects voting when poll has already ended (2087ms)
 rejects voting when poll was disabled later via `setPollStatus()` (2093ms)
 fails with wrong poll PDA (1675ms)

Unit: `setPollStatus()`

admin can toggle poll status (1248ms)
 non-admin cannot toggle poll status (870ms)
 fails `if poll PDA is wrong` (not initialized / seeds mismatch) (392ms)
 fails `if caller` signs but `caller` account is not the admin pubkey (858ms)

Units: `commitRewardsRoot()`

commits the root, funds the vault and snapshots the claim deadline (3330ms)
 consumes the daily budget once, `for` the whole committed total (3355ms)
 rejects a commit before the poll has ended (2519ms)
 rejects a zero root or a zero total (2940ms)
 rejects a `caller` that is neither admin nor rewarder (4185ms)
 lets the rewarder commit as well as the admin (5439ms)
 admin can replace a root before any claim, paying only the increase (3774ms)
 a downward correction charges nothing and leaves the surplus `for` the sweep (3770ms)
 the rewarder cannot replace a root, only the admin can (5448ms)
 works on a disabled poll: a ban does not cancel rewards already earned (3773ms)
 a correction cannot shorten a deadline already promised (4615ms)
 a correction may still extend a deadline (5450ms)
 refuses to replace a root once the first claim has landed (3783ms)

Units: `claimReward()`

pays the committed amount and marks the vote claimed (3767ms)
 can be executed by anyone on the voter behalf (5029ms)
 rejects an amount that is not the committed one (3362ms)
 rejects a proof belonging to a different voter (5864ms)
 rejects a proof from another poll (7549ms)
 rejects a proof with a missing or an extra sibling (5453ms)
 a recipient duplicated `in` the committed `set` can still claim only once (3784ms)
 rejects a second claim (3769ms)
 rejects a claim before anything is committed (2518ms)
 an underfunded reward vault fails the claim and leaves deposits untouched (3766ms)
 closes the vote and returns its rent once the deposit is already out (5452ms)

units: `withdraw`

pays the deposit back once the poll has ended, with no reward involved (2934ms)
 rejects a withdrawal before the poll has ended (2103ms)
 rejects a second withdrawal (2934ms)
 still pays out when the poll was banned (3352ms)
 keeps the vote open `while` a reward is still claimable (4190ms)
 closes the vote and refunds its rent when the reward was already claimed (4199ms)
`withdraw_promo` pays `promo_claimable` from the promo vault and zeros it (2515ms)

Unit: `calculatePollFee()`

emits `PollFeeCalculated` with correct fee (no discount at exactly 1 day) (469ms)
 emits `PollFeeCalculated` with correct fee (10% discount after 1 day + 1 second) (413ms)
 update `price_per_second` from `systemCore` (1215ms)
 discount uses integer math (floor) on non-even `base_cost` (rounding) (1258ms)
 computes a discount on a base cost above `u64::MAX / 10` (1259ms)
 rejects invalid `time` range (end <= start)

Unit: `promo funder + distributePromoReward()`

admin can `set` promo funder (2090ms)
 admin can update promo funder repeatedly (3353ms)
 non-admin cannot `set` promo funder (UnauthorizedAdmin) (2941ms)
 fails `setPromoFunder` with wrong `promoConfig` PDA (1257ms)
 fails `setPromoFunder` with wrong `systemCore` PDA (1259ms)
 promo funder can distribute promo reward: moves tokens to `system_core_ata` + credits recipient profile (3771ms)
 admin can distribute promo reward even when promo funder is someone `else` (3353ms)

random user cannot distribute promo reward (UnauthorizedRewarder) (4617ms)
 fails **if** promo callerAta.owner != **caller** (2943ms)
 fails **if** promo callerAta.mint != mint (4001ms)
 fails distributePromoReward with wrong systemCoreAta PDA (2708ms)
 rejects attempt to send promo with mismatched userProfile account (4612ms)
 recipient can withdraw promo reward to ATA (4186ms)
 rejects zero amount (3359ms)

unit: daily reward budget

admin can **set** budget and non-admin cannot (1673ms)
 accumulates reward commitments within configured daily budget (5450ms)
 allows exact remaining budget and rejects above remaining budget without state changes (5028ms)
 allows lowering budget below current spent and **then** blocks further same-day commitments (5452ms)
 allows zero budget and disables commitments (3772ms)
 accepts u64::MAX budget **for** normal commitments (4189ms)
 uses one shared daily budget **for** reward commitments and promo payouts (8384ms)
 rejects a second commitment above the remaining daily budget (5033ms)
 rejects promo payout above the remaining daily budget (6710ms)

Unit: account versioning

current version on system core at initialize
 current version on poll + creator profile at createPoll (418ms)
 current version on vote + voter profile at makeVote (2101ms)
 current version on promo config at setPromoFunder (1678ms)

Units: Merkle vectors (shared with the Rust implementation)

produces the committed leaf hashes
 produces the committed roots **for** the 1, 2, 3 and 5 leaf trees
 produces the committed proofs, and they verify
 hashes node pairs **in** sorted order, so proofs need no direction bits
 separates the leaf and node domains
 binds a leaf to its poll, recipient, amount and kind

Units: claimCreatorReward(), closeRewardVault(), closeVote()

pays the creator their committed leaf, once (3781ms)
 rejects a creator amount that is not the committed one (3355ms)
 a voter cannot redeem the creator leaf through the voter path (3364ms)
 the creator cannot redeem a voter leaf through the creator path (3770ms)
 the creator can vote on their own poll and collect both leaves (3355ms)
 rejects closing the reward vault before the claim window has passed (4623ms)
 closeVote rejects a vote that is not settled (4617ms)

Units: poll duration limits

defaults to 30 days at initialize
 rejects a poll shorter than one day, and accepts one exactly a day long (411ms)
 still reports an inverted range as an invalid **time** range, not a short poll
 accepts a poll exactly at the limit and rejects one second more (419ms)
 admin can change the limit, and it applies to the next poll (1252ms)
 rejects a limit below one day, so poll creation cannot be switched off by a typo
 rejects a limit above one year, the symmetric half of the floor
 non-admin cannot change the limit (1246ms)
 does not disturb polls that already exist (1254ms)

Units: closePollVault()

refunds the vault rent to the creator once every deposit is out (5901ms)
 rejects closing **while** the poll is still running (1659ms)
 rejects closing **while** a deposit is still **in** the vault (4605ms)
 a stray transfer into the vault keeps it open (2521ms)
 pays the rent to the creator, not to whoever is passed as creator (2100ms)

Units: setClaimWindow()

rejects a window shorter than one day
 rejects zero, which is the shape a missing argument takes
 accepts exactly one day, the boundary itself (814ms)
 rejects a window longer than a year, and accepts exactly a year (836ms)
 cannot be changed by a non-admin (1266ms)

FLOW: poll lifecycle with committed rewards

carries a poll from votes through claims to closed accounts (8803ms)
 settles a claim and a deposit **in** one transaction, with one signature (4196ms)
 settles a creator voter leaf, creator leaf and deposit **in** one transaction (2540ms)
 creates the voter's canonical ATA **in** the same transaction as the claim (3752ms)
 lets the voter fund their own ATA, claim and withdraw on one signature (3792ms)

FLOW: concurrent votes across polls run **in** parallel
concurrent votes (different polls) -> succeeded: 4, failed: 0 (of 4)
all 4 simultaneous votes on 4 different polls succeed (7111ms)
concurrent votes (same poll, distinct voters) -> succeeded: 4 (of 4)
multiple distinct voters on the SAME poll can vote concurrently (5866ms)

FLOW: votes within a single poll are NOT parallel
same-poll votes share 2 writable account(s) -> cannot run **in** parallel
two votes on the SAME poll conflict on writable accounts (poll + poll vault) (2936ms)
different-poll votes share 0 writable account(s) -> parallelizable
two votes on DIFFERENT polls have disjoint writable accounts (contrast) (3356ms)

FLOW: concurrent poll creation no longer collides
concurrent createPoll (distinct creators, same nonce) -> succeeded: 4, failed: 0 (of 4)
all 4 simultaneous createPoll calls succeed (even reusing nonce across creators) (5475ms)
same creator can create distinct polls with different nonces (2070ms)
supports nonce edge values: 0 and u64::MAX (3359ms)
a creator reusing the same nonce collides on init (one address per (creator, nonce)) (1680ms)

FLOW: createPoll writable-account analysis
two createPoll calls share 1 writable account(s) (the fee ATA) -> creations succeed but serialize on fees
two creations lock distinct poll + poll vault, and system_core is no longer writable (2510ms)

180 passing (7m)

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.